

Geo-lucid Conditional Diffusion Models for High Physical Fidelity Trajectory Generation

Mingzhou Yang
University of Minnesota
Minneapolis, USA
yang7492@umn.edu

Arun Sharma
University of Minnesota
Minneapolis, USA
sharm485@umn.edu

Majid Farhadloo
University of Minnesota
Minneapolis, USA
farha043@umn.edu

Bharat Jayaprakash
University of Minnesota
Minneapolis, USA
jayap015@umn.edu

Shashi Shekhar
University of Minnesota
Minneapolis, USA
shekhar@umn.edu

Abstract

Given a set of historical vehicle trajectories and their descriptive attributes, the goal is to train a generative model that produces synthetic trajectories with high physical fidelity. Here, physical fidelity is defined as fidelity to both geometric and dynamic properties of trajectories. The problem is important since trajectory generation can contribute to data augmentation for many traffic-related applications, such as popular route discovery and traffic light control. The key challenge of this problem lies in achieving high physical fidelity under coarse geospatial attributes (e.g., origin-destination pairs) that lack fine-grained details. Current methods, which mostly focus on geometric properties, have limited utility in domain-specific scenarios due to their neglect of trajectory dynamics. To address these limitations, we propose GCDM, a novel Geo-Lucid Conditional Diffusion Model framework that integrates road map attributes into the generative process through spatially hierarchical generation and map-informed latent variables. Experiments on real-world vehicle trajectory datasets show that GCDM outperforms state-of-the-art methods in geo-distribution similarity and dynamics fidelity.

CCS Concepts

• **Information systems** → **Geographic information systems**; • **Applied computing** → **Transportation**.

Keywords

Geo-lucid Neural Network, Physical Fidelity, Trajectory Generation, Urban Computing

ACM Reference Format:

Mingzhou Yang, Arun Sharma, Majid Farhadloo, Bharat Jayaprakash, and Shashi Shekhar. 2025. Geo-lucid Conditional Diffusion Models for High Physical Fidelity Trajectory Generation. In *The 33rd ACM International Conference*

on Advances in Geographic Information Systems (SIGSPATIAL '25), November 3–6, 2025, Minneapolis, MN, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3748636.3762749>

1 Introduction

We aim to train a geo-lucid generative model that synthesizes trajectories with high physical fidelity based on attributed road maps (e.g., OpenStreetMap [64]), given historical trajectories and their descriptive attributes (e.g., origin/destination pairs, departure time slots). High physical fidelity, here, means capturing the geometric (e.g., spatial distribution) and dynamic (e.g., velocity patterns) properties of real-world trajectories. A spatial-temporal trajectory is defined as a time-ordered sequence of GPS coordinates $\{(lat_i, lon_i, t_i)\}_{i=1}^n$ describing object movement in geographical space. The generation of such data is important for societal applications across urban domains. For example, trajectory data underpins intelligent transportation systems, traffic optimization, and congestion prediction [24, 56]. It enables analysis of urban dynamics, hotspot identification, and resource allocation improvements [2, 17]. Continuous trajectory generation is also beneficial for location-based services (e.g., ride-sharing, food delivery). Generated trajectories can support downstream tasks such as improved estimated time-of-arrival predictions and refined travel energy estimation in regions with insufficient measurements. Furthermore, synthesizing trajectories offers advantages over directly using real-world data by mitigating fundamental constraints related to commercial restrictions on proprietary datasets, pervasive missing values, and the prohibitive costs associated with data collection [25, 66].

Prior work on trajectory generation via deep generative models can be broadly categorized into discrete and continuous approaches. In discrete settings, trajectories are commonly represented as discretized images [3, 34], sequences of grids [13, 30, 37], road segments [7, 23, 60], or points of interest (POIs) [10, 54, 61]. However, these representations inherently compromise the fidelity and granularity of the generated trajectories, largely due to their reliance on low spatiotemporal resolution representations [38]. Recent research has increasingly focused on continuous trajectory generation, specifically, approaches based on generative adversarial networks (GANs) [36, 45], variational autoencoders (VAEs) [6, 70], and diffusion models [62, 72, 73]. These methods primarily focus on the *geometric properties* of trajectories to optimize their alignment with the geographic distributions of the ground truth. However, a

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGSPATIAL '25, Minneapolis, MN, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2086-4/2025/11
<https://doi.org/10.1145/3748636.3762749>

critical gap remains in modeling **trajectory dynamics**, which are essential for real-world applications of the generated trajectories. Trajectory dynamics here is defined as physical properties including velocity (the first-order derivative of the displacement-time function), acceleration (the second-order derivative), and jerk (the third-order derivative).

It is important that generative models respect dynamic properties for realistic data generation. Considering these properties is essential for employing synthetically generated trajectories for downstream applications, as they work to preserve physical fidelity. For example, effective traffic-flow optimization requires realistic acceleration and deceleration profiles to simulate congestion accurately. However, trajectory attributes are typically limited to coarse geospatial descriptors (e.g., total distance and mean velocity), thereby discarding critical micro-scale features such as turn-by-turn route details and instantaneous velocity profiles. Together, these omissions create significant challenges for generating spatiotemporal trajectories with high physical fidelity under coarse geospatial constraints.

To overcome these limitations, we propose a novel Geo-lucid Conditional Diffusion Model (GCDM) framework that explicitly incorporates the semantic and dynamic attributes of a road map into the generative process, as illustrated in Figure 1. Unlike traditional diffusion models that corrupt the input data into random noise (e.g., Gaussian) during the forward process and then reconstruct it via denoising in the reverse process (Figure 1a), we propose a spatial residual diffusion method based on trajectory decomposition that injects road map priors into the latent variables (i.e., the initial state of the diffusion process shown in Figure 1b). We demonstrate that the rich information contained in road map attributes can guide both the geometric and dynamic properties of the generated trajectories. Our contributions are as follows:

- We introduce a cascaded diffusion model framework that leverages a spatial hierarchy to decouple trajectory generation into two stages: coarse trajectory synthesis followed by high-resolution GPS trajectory generation.
- We explicitly incorporate an attributed road map into the generative process at multiple granularities using two techniques: map-informed latent variables (GPS level) and trajectory projection (coarse level).
- Through extensive experiments on real-world vehicle trajectory datasets, we demonstrate that GCDM outperforms state-of-the-art methods in both geo-distribution similarity and dynamic fidelity.

Scope: In this work, we address the problem of generating trajectories when only coarse-level descriptive attributes and background road maps are available. Tasks such as synthesizing trajectories from partial trajectories [32, 47, 63], turn-by-turn travel routes [74], or drive cycles are beyond the scope of this paper, as explained in Section 5. Advancing privacy preservation methods of trajectory data falls outside the scope of the paper.

2 Preliminaries

2.1 Notations

Definition 2.1. A **trajectory** x of length n is defined as a sequence of spatiotemporal points $\{p_1, p_2, \dots, p_n\}$, where each point $p_i =$

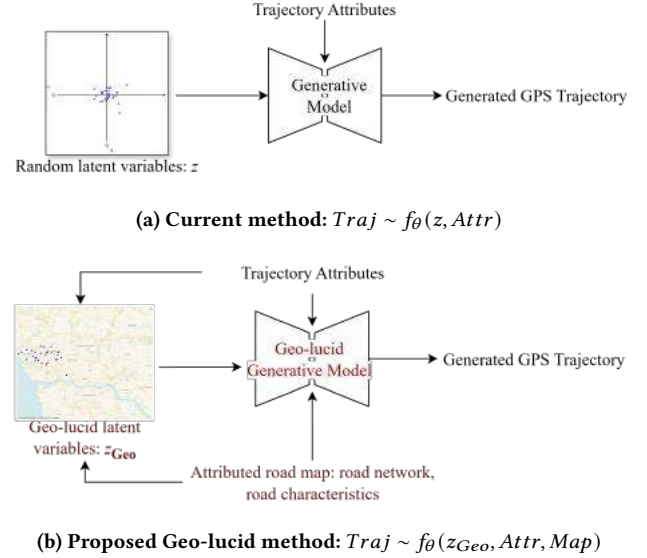


Figure 1: Comparison of trajectory generation methods. Our geo-lucid method explicitly incorporates attributed road maps into the generation framework. $f_{\theta}(\cdot)$ denotes the generative models.

(lat_i, lon_i, t_i) represents a GPS coordinate at discrete time step $t \in \{1, 2, \dots, n\}$. The time interval between consecutive GPS samples is fixed at ϵ seconds, corresponding to a sampling rate of $1/\epsilon$. A **trajectory dataset** $\mathcal{X}$ is a collection of such trajectories, where each trajectory x belongs to $\mathcal{X}$.

Definition 2.2. A generated trajectory dataset $\tilde{\mathcal{X}}$ is said to exhibit **high physical fidelity** if both its geometric properties (i.e., geographic distribution) and its dynamic properties (i.e., velocity, acceleration, jerk) closely approximate those of the ground truth trajectory dataset $\mathcal{X}$.

Definition 2.3. **Trip description attributes** refer to the inherent properties associated with an entire trajectory. In this paper, these attributes include the origin/destination pairs, departure time, travel duration, total distance, and average speed.

Definition 2.4. An **attributed road map** is a geo-referenced representation of a road network that integrates both spatial and attribute data [52]. For example, OpenStreetMap, a widely used road map with attributes, not only encodes the connectivity of road networks but also provides detailed information such as the GPS coordinates of road segments and intersections, road types, speed limits, turn restrictions, and other relevant attributes [64].

Definition 2.5. A **geo-lucid trajectory generation model** explicitly incorporates the attributed road map into the generative framework. The generation process is defined as $\tilde{x} \sim f_{\theta}(z, Attr, Map)$, where $f_{\theta}(\cdot)$ denotes the generation model; z denotes the latent variables; $Attr$ refers to the trajectory attributes.

2.2 Problem Definition

The trajectory generation problem is defined as follows:

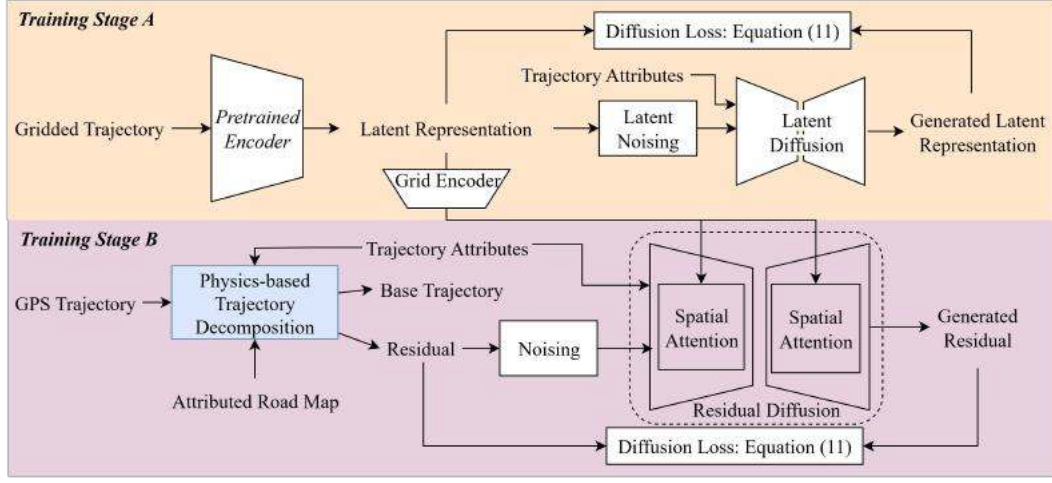


Figure 2: Training objectives of GCDM.

- **Inputs:** A trajectory dataset $\mathcal{X}$ and its description attributes
- **Output:** A geo-lucid generative model $f_\theta(\cdot)$ capable of producing a synthetic trajectory dataset $\tilde{\mathcal{X}}$ conditioned on specified trajectory attributes
- **Objectives:** Solution quality (high physical fidelity)
- **Constraints:** Trajectory locations are represented in a continuous spatial domain.

The problem is to develop a generative model that learns the complex spatiotemporal dynamics of movement from a trajectory dataset and its corresponding road map, to achieve high physical fidelity in the generated data. In this work, we consider both the similarity of geo-distribution and dynamic properties to assess the physical fidelity of the generated trajectories. Therefore, we focus on the problem of continuous GPS trajectory generation, in which the dynamic patterns possess significant physical meaning. We use the distribution shift of velocity, acceleration, and jerk as a proxy for the dynamic fidelity of generated trajectories.

2.3 Conditional Diffusion Probabilistic Model

In a traditional conditional diffusion model [20, 67, 71], isotropic Gaussian noise is progressively added to the data samples through a predetermined Markov chain, denoted by $q(\mathbf{x}_t | \mathbf{x}_{t-1})$. This process generates a series of intermediate noisy data referred to as *forward* diffusion. Conversely, in *reverse* diffusion, the process begins with completely noisy data, $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The model then incrementally recovers the original data distribution conditioned on a conditional variable $\mathbf{y}$ through successive iterations ($\mathbf{x}_{T-1}, \mathbf{x}_{T-2}, \dots, \mathbf{x}_0$).

Forward Process (q): In this process, Gaussian noise is progressively added to $\mathbf{x}_0$ over T iterations:

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}), \quad (1)$$

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t | \sqrt{\alpha_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (2)$$

Here, the hyperparameters $\beta_{1:T}$ are constrained between 0 and 1, representing the noise variance introduced at each iteration, and

$\alpha_t = 1 - \beta_t$. The variable $\mathbf{x}_{t-1}$ is scaled down by a factor of $\sqrt{\alpha_t}$ to keep the variance of the random variables finite. Additionally, the derivation of $\mathbf{x}_t$ from $\mathbf{x}_0$ can be streamlined using Equation (3):

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t | \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad (3)$$

where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. In addition, one can derive the posterior distribution of $\mathbf{x}_{t-1}$ given $(\mathbf{x}_0, \mathbf{x}_t)$ as

$$q(\mathbf{x}_{t-1} | \mathbf{x}_0, \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1} | \boldsymbol{\mu}, \sigma^2 \mathbf{I}),$$

$$\boldsymbol{\mu} = \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t, \quad \sigma^2 = \frac{(1 - \bar{\alpha}_{t-1}) \beta_t}{1 - \bar{\alpha}_t}. \quad (4)$$

Reverse Process (p_θ): In this process, we leverage an additional conditional variable $\mathbf{y}$ and optimize a neural denoising model f_θ that takes as input the condition $\mathbf{y}$ and a noisy state $\tilde{\mathbf{x}}$,

$$\tilde{\mathbf{x}} = \sqrt{\bar{\alpha}} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (5)$$

where $\boldsymbol{\epsilon}$ is sampled from a standard normal distribution. Equation (5) represents the noisy state $\tilde{\mathbf{x}}$, which follows the marginal distribution of noisy states at different stages of the forward diffusion process described in (3). The objective is to recover the original noise-free target state $\mathbf{x}_0$.

In contrast to the forward process q , the reverse process p_θ follows a backward trajectory, starting from Gaussian noise $\mathbf{x}_T$:

$$p_\theta(\mathbf{x}_{0:T} | \mathbf{y}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{y}), \quad (6)$$

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T | \mathbf{0}, \mathbf{I}), \quad (7)$$

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{y}) = \mathcal{N}(\mathbf{x}_{t-1} | \mu_\theta(\mathbf{y}, \mathbf{x}_t, t), \sigma_t^2 \mathbf{I}). \quad (8)$$

The inference process is defined using learned isotropic Gaussian conditional distributions $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{y})$. The denoising model f_θ is then designed to predict $\boldsymbol{\epsilon}$ from any noisy state $\tilde{\mathbf{x}}$, including $\mathbf{x}_t$. This allows us to estimate $\mathbf{x}_0$ by rearranging Equation (5) as:

$$\hat{\mathbf{x}}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} f_\theta(\mathbf{y}, \mathbf{x}_t, t) \right). \quad (9)$$

We then substitute $\hat{\mathbf{x}}_0$ into the posterior distribution of $q(\mathbf{x}_{t-1} | \mathbf{x}_0, \mathbf{x}_t)$, parameterizing the mean of $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{y})$ as in Equation (10). The

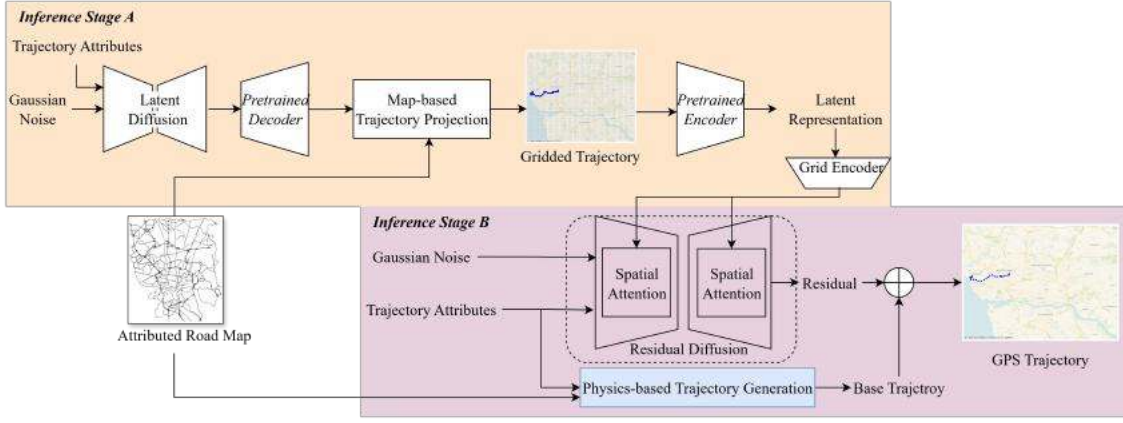


Figure 3: Inference architecture for GCDM.

variance of $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{y})$ follows the variance of the forward process:

$$\mu_\theta(\mathbf{y}, \mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} f_\theta(\mathbf{y}, \mathbf{x}_t, t) \right), \text{ and } \sigma_t = \sqrt{\tilde{\beta}_t}, \quad (10)$$

where $\tilde{\beta}_t = \frac{(1-\alpha_{t-1})\beta_t}{1-\alpha_t}$ ($t > 1$) and $\tilde{\beta}_1 = \beta_1$.

The goal of training a diffusion model is to minimize the mean square error between the model-predicted noise and the Gaussian noise ϵ :

$$\min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \mathbb{E}_{\mathbf{x}_0, \mathbf{y}, \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - f_\theta(\mathbf{y}, \mathbf{x}_t, t)\|_2^2], \quad (11)$$

where $\mathbf{y}$ denotes the condition and t is the diffusion step.

3 Proposed Approach

The overall training and sampling process of the proposed Geolucid Conditional Diffusion framework (GCDM) is illustrated in Figures 2 and 3. The framework decouples trajectory generation into two spatially hierarchical stages, both implemented as deep neural networks. In stage A, we first generate trajectories at a coarse level by representing each trajectory as a binary image. In stage B, we refine these into high-resolution GPS trajectories conditioned on their coarse counterparts. The two stages are trained in parallel. This cascaded framework offers two key advantages over training a single-stage model at the highest resolution. First, as observed in cascaded pipelines across other domains [21, 40, 43, 46], the low-resolution generation can provide extra guidance towards the final results. Second, the hierarchical design enables the integration of an attributed road map at multiple spatial granularities. In this work, we incorporate (1) road network projection (coarse-level constraints) and (2) map-informed latent variables (GPS-level features) into the framework. Detailed training procedures are provided in Appendix A.

3.1 Stage A

In Stage A, we first convert each trajectory in the training dataset into a binary image through a gridding process. Let the geographic study area be defined by a bounding box with longitude and latitude

ranges $[lon_{min}, lon_{max}]$ and $[lat_{min}, lat_{max}]$, respectively. Given a grid resolution of $N \times N$, each GPS coordinate (lon, lat) in the trajectory dataset is mapped to a grid cell $(grid_x, grid_y)$ using:

$$grid_x = \left\lfloor \frac{(lon - lon_{min}) \cdot N}{lon_{max} - lon_{min}} \right\rfloor, \quad grid_y = \left\lfloor \frac{(lat - lat_{min}) \cdot N}{lat_{max} - lat_{min}} \right\rfloor, \quad (12)$$

where $\lfloor \cdot \rfloor$ denotes the floor function. The study area is then represented as an $N \times N$ binary matrix (image), where each cell (pixel) corresponds to a grid. Cells traversed by the trajectory are assigned a value of 1 (black), and all others are 0 (white).

A well-established technique for reducing computational burden is compressing data into lower-dimensional representations, particularly in image generation [4, 48]. Following this paradigm, Stage A employs a latent diffusion model [49] over compressed representations of trajectory images instead of raw pixel space. To achieve this, we pre-train a variational autoencoder (VAE) [53] with an encoder that compresses a binary image of shape $(1, N, N)$ into a latent tensor (C, H, W) ($H, W \ll N$), and a decoder that reconstructs the original image. To address the extreme class imbalance caused by sparse trajectory-occupied pixels, we use Focal Loss [51] as the reconstruction loss in VAE. For a binary image $\mathbf{X}$ and reconstruction $\hat{\mathbf{X}}$, the reconstruction loss is defined as:

$$\mathcal{L}_{Rec} = \frac{1}{N^2} \sum_{i,j} \alpha(1 - p_t^{(i,j)})^\gamma \text{BCE}(x_{i,j}, \hat{x}_{i,j}), \quad (13)$$

where $p_t^{(i,j)} = \hat{x}_{i,j}$ if $x_{i,j} = 1$ (black) else $1 - \hat{x}_{i,j}$, and α, γ are tunable parameters. Here, $\text{BCE}(x, \hat{x}) = -x \log(\hat{x}) - (1-x) \log(1-\hat{x})$ is the binary cross-entropy loss. Finally, we train a latent diffusion model [49] conditioned on the trajectory description attributes using the pre-trained VAE encoder, as shown in Figure 2.

3.2 Stage B

Stage B generates GPS trajectories conditioned on their coarse counterparts and trajectory attributes. Unlike existing diffusion models for trajectory generation (e.g., [73]), which rely entirely on embeddings of trajectory attributes, the proposed method augments the generation framework with geospatial knowledge through two

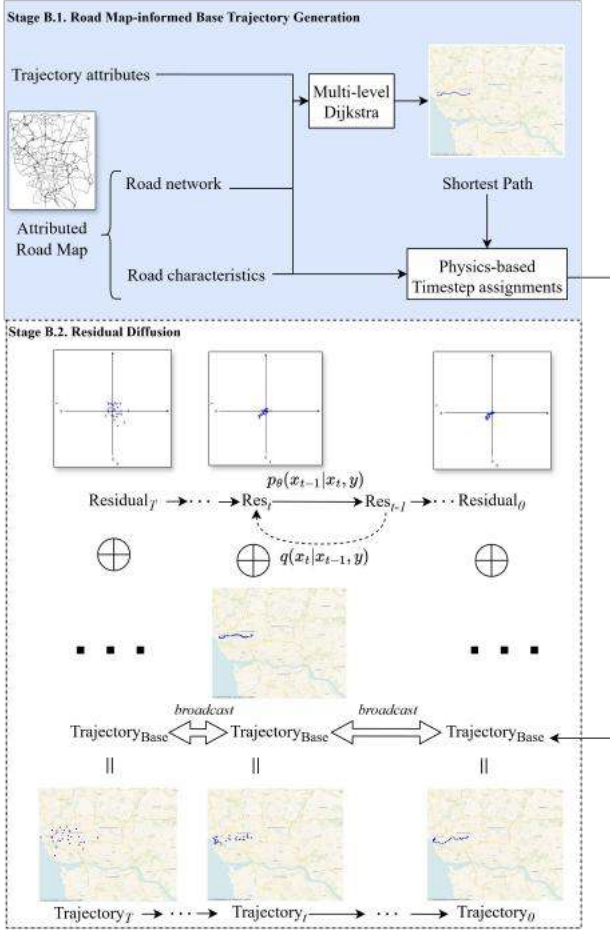


Figure 4: (Stage B) Residual diffusion based on road map-informed trajectory decomposition/composition.

key components: (1) Map-informed latent variables and (2) Spatial-attention mechanisms for coarse trajectory guidance.

3.2.1 Geo-lucid map-informed latent variables. Traditional diffusion models, as described in Section 2.3, corrupt the input data into random noise (e.g., Gaussian) during the forward process and then reconstruct it via denoising in the reverse process. However, such noise corruption disrupts the intrinsic geospatial distribution of the original trajectory, forcing the generation process to rely solely on coarse trajectory attributes to maintain physical fidelity. To address this limitation, we propose a spatial residual diffusion method based on trajectory decomposition that injects road map priors into the latent variables (i.e., the initial state of the diffusion process). Unlike traditional diffusion, which perturbs the entire trajectory, our approach decomposes a trajectory into two components:

- A **base trajectory** derived directly from trajectory attributes and the attributed road map via a physics-based model
- A **residual deviation** that captures fine-grained spatial variations

Figure 4 illustrates the process of residual diffusion based on physics-based trajectory decomposition. Given the origin and destination nodes, the base trajectory is computed in two stages. First, the shortest path along the road network is determined using a multi-level Dijkstra algorithm [9], and the corresponding GPS sequence is extracted from the road map. Next, the dynamic property of the base trajectory, i.e., the timestamp for each GPS point, is computed based on the estimated travel duration for each road segment using a physics-based model. Specifically, the travel duration for a road segment is computed as [39]:

$$\text{Duration} = \left(\frac{\text{Length}}{V_{\max} \cdot \text{Reduction}} \right) \times \text{Penalty}, \quad (14)$$

where $V_{\max}$ denotes the speed limit of the segment, Reduction is a factor accounting for conditions that effectively reduce the maximum speed. The Penalty factor adjusts for various road characteristics and is determined as the maximum among the following [39]:

- **Service Penalty**, which reflects constraints associated with service roads, which often have lower speed limits or additional restrictions.
- **Width Penalty**, which accounts for the impact of road width on travel efficiency, with narrower roads incurring higher penalties.
- **Alternating Route Penalty**, which applies to reversible roads.
- **Sideroad Penalty**, which represents the increased travel time associated with side roads.

The timestamps for individual GPS points along the shortest path are then obtained by linearly interpolating the durations of the corresponding road segments, thereby generating the base trajectory. To further balance the reliance on map-informed priors versus learned residuals, we introduce a flexible decomposition factor controlled by a parameter λ :

$$\text{Residual}_t = \frac{\text{Traj}_t - \lambda \cdot \text{Traj}_{\text{base}}}{1 - \lambda}, \quad 0 \leq \lambda < 1. \quad (15)$$

Setting $\lambda = 0$ recovers the traditional diffusion model (with no map-informed priors) while letting $\lambda \rightarrow 1$ imposes a strong assumption that the trajectory closely follows the base trajectory. This tunable integration maintains flexibility while leveraging roadmap information. Finally, the diffusion process is applied *only* to the residual. During generation, the final trajectory is reconstructed by combining the base trajectory with the generated residual.

3.2.2 Residual Diffusion with Spatial Guidance. The proposed method integrates geospatial context into trajectory generation through a spatial attention mechanism. Building on a U-Net architecture [50, 73], we condition the diffusion process on coarse trajectory features encoded into spatial tokens. This enables the model to refine trajectories using localized geographic priors.

Specifically, given a latent 2D grid $z_{2d} \in \mathbb{R}^{C \times H \times W}$, a grid encoder transforms it into spatial tokens $\mathbf{M} \in \mathbb{R}^{N \times d}$ through strided convolutions and adaptive pooling:

$$\mathbf{M} = \text{Flatten}(\text{AdaptPool}(\text{Conv}(z_{2d}))) + \text{Pos}, \quad (16)$$

where $\text{Conv}(\cdot)$ denotes convolutional downsampling, AdaptPool reduces spatial resolution to a fixed low-resolution grid, and $\text{Pos} \in$

$\mathbb{R}^{N \times d}$ are learnable positional embeddings that preserve spatial relationships. This produces resolution-agnostic tokens that encode the local map structure. Furthermore, during Stage B training, we intermittently add noise to the latent 2D grid z_{2d} to teach the model to understand non-perfect embeddings with Stage A [21].

The tokens guide trajectory generation via cross-attention at multiple stages in the U-Net’s downsampling and upsampling paths. For a hidden state $\mathbf{h} \in \mathbb{R}^{CH \times L}$ (channels $\times$ Length), queries $\mathbf{q}$ are derived through a 1×1 convolution, while keys and values are mapped from $\mathbf{M}$:

$$\mathbf{q} = \text{Proj}_Q(\mathbf{h}), \quad \mathbf{q} \in \mathbb{R}^{L \times d}, \quad (17)$$

$$\mathbf{o} = \text{MultiHead}(\mathbf{q}, \mathbf{M}, \mathbf{M}), \quad (18)$$

$$\mathbf{h}' = \mathbf{h} + \text{Proj}_{\text{out}}(\mathbf{o}), \quad (19)$$

where Proj_{out} projects the attention output back to the U-Net’s channel dimension. This allows temporal features in $\mathbf{h}$ to dynamically focus on relevant spatial regions.

The enhanced hidden states $\mathbf{h}'$ are processed by subsequent layers in the conditional U-Net [73], which concurrently embeds trajectory attributes and diffusion time steps. By injecting spatial guidance without altering the core denoising mechanism, the method preserves the U-Net’s efficiency while improving guidance to geographic priors.

3.3 Road Map-Informed Trajectory Sampling

Figure 3 illustrates GCDM’s two-stage sampling pipeline. During Stage A, we generate a latent representation of coarse trajectories from Gaussian noise $\mathcal{N}(0, \mathbf{I})$ and trajectory attributes. The latent representation is decoded to a binary image $\mathbf{G} \in \{0, 1\}^{H \times W}$, where $G_{i,j} = 1$ indicates a trajectory presence at cell (i, j) . To enforce road network constraints, we refine $\mathbf{G}$ with road map alignment. Specifically, we first generate a road mask $\mathbf{R}$ by coloring each grid that is covered by road segments in black (value 1). We then project off-road trajectory cells ($G_{i,j} = 1$ where road mask $R_{i,j} = 0$) to their nearest road cells (k, l) using their Euclidean distance. The refined image $\mathbf{G}'$ retains on-road cells and replaces off-road cells with their nearest road projections. Then, $\mathbf{G}'$ is re-encoded into the latent space to guide Stage B, where the final GPS trajectory Traj_0 is synthesized by combining:

$$\text{Traj}_0 = \underbrace{\lambda \cdot \text{Traj}_{\text{base}}}_{\text{Road map-derived}} + \underbrace{(1 - \lambda) \cdot \text{Residual}_0}_{\text{Generated deviation}}, \quad (20)$$

Here, $\text{Traj}_{\text{base}}$ is derived from trajectory attributes and road map as described in Sec 3.2.1, and Residual_0 is generated by our residual diffusion model conditioned on $\mathbf{G}'$ and trajectory attributes. The parameter $\lambda \in [0, 1)$ balances the map-informed priors against the learned residuals.

4 Evaluation

Experiment Goals. We conducted a comprehensive set of experiments using real-world data to evaluate our GCDM model. The experiments were designed to answer three main questions:

- **RQ1:** How does the quality of trajectories generated by our method compare to those generated by state-of-the-art (SOTA) approaches?

- **RQ2:** How do individual components of our model affect trajectory quality?
- **RQ3:** How does the two-stage architecture of GCDM affect the trade-off between computational cost and physical fidelity in synthetic trajectory generation?

The comparative analysis considered both the geo-distribution and physical fidelity of generated trajectories. The sensitivity analysis included an ablation study of the incorporation of road maps and a test of the effect of grid size on solution quality. The last set of experiments was essentially a time complexity analysis.

4.1 Experimental Setup

Data. We assessed the performance of GCDM and several competitive baselines on two open-source real-world vehicle trajectory datasets. The road maps for both datasets are extracted from OpenStreetMap [64] based on the lon/lat bounding box corresponding to the datasets. The first dataset comprises trajectories recorded by 442 taxis operating in Porto, Portugal, over one year (from July 2013 to June 2014)¹. In total, 1,411,784 trajectories were captured with a fixed 15-second sampling interval. The second dataset contains 1,250,071 trajectories collected from 1,3000 taxis in Harbin, China [31]. The sampling interval for the Harbin data was 30 seconds. Detailed preprocessing procedures and additional dataset statistics are provided in Appendix B.

Comparison Approaches. We compared our approach against four baseline methods set up per their respective code repositories [14, 26, 70, 73].

- EETG [70], a VAE-based trajectory generation method that combines a VAE with a VRNN [8] to encode and generate trajectories using an LSTM.
- Diffwave [26], a method built on the WaveNet architecture and specifically designed for synthesizing sequential data, including time series and speech signals.
- Sashimi [14], a multi-scale architecture for waveform modeling that leverages the S4 model [15] to effectively capture long-range dependencies in sequence data.
- DiffTraj [73], the current state-of-the-art trajectory generation framework based on diffusion models. It employs a U-Net backbone and conditions the generation of trip attributes via a wide and deep module.

Evaluation Metrics. We evaluate both the geo-distribution and the dynamic fidelity of generated trajectories. Full details are available in Appendix B. For geo-distribution similarity, we adhered to established practices [11, 73, 74]. Specifically, we employed three quantitative metrics to assess the fidelity of the generated trajectories: (i) *Density error*, which quantifies the discrepancy between the spatial distributions of generated and real trajectories, (ii) *Trip error*, which evaluates differences in the distributions of trip origins and destinations, and (iii) *Pattern score*, which measures the similarity in trajectory patterns. We utilized the Jensen-Shannon divergence (JSD) formula to compare the distributions P (generated) and Q (real). The JSD is defined as:

$$\text{JSD}(P||Q) = \frac{1}{2}D_{\text{KL}}(P||M) + \frac{1}{2}D_{\text{KL}}(Q||M), \quad (21)$$

¹<https://www.kaggle.com/datasets/craita/taxi-trajectory>

where $M = \frac{1}{2}(P + Q)$ and D_{KL} denotes the Kullback-Leibler divergence [28].

For dynamic fidelity, we quantified the distributional shifts in velocity, acceleration, and jerk profiles using three metrics: JSD, Wasserstein distance, and the Kolmogorov-Smirnov (KS) statistic. To compute the JSD, we partitioned the profiles into 50 bins and derived the corresponding discretized distribution. The Wasserstein distance is defined as:

$$W_1(P, Q) = \int_{\mathbb{R}} |F_P(x) - F_Q(x)| dx, \quad (22)$$

where $F_P(x) = P(X \leq x)$ and $F_Q(x) = Q(X \leq x)$ are the respective cumulative distribution functions. The KS statistic is given by:

$$KS(P, Q) = \sup_x |F_P(x) - F_Q(x)|. \quad (23)$$

Implementation Details. The GCDM model was implemented using standard neural network components with specific parameter settings detailed in Appendix A. Our codes are available online². All experiments were conducted using Python 3.12 and PyTorch 2.4 on a single NVIDIA A100 40GB GPU.

4.2 RQ1: Comparative Analysis

4.2.1 Geo-Distribution Similarity. We evaluated the geometric properties of the generated trajectories using the three geo-distribution similarity metrics described in Sec. 4.1. Table 1 shows that the proposed method outperforms all baselines on both datasets across most metrics. For example, on the Porto dataset, GCDM reduces the density error by 13% compared to the previous best method (DiffTraj). The 4.3% improvement in trip error also demonstrates GCDM’s ability to preserve geometric properties. On the Harbin dataset, GCDM is 23% better on density error compared with the existing state-of-the-art. These results underscore the effectiveness of incorporating the road map to generate trajectories that reflect the geometric properties of real-world trips. A visualization of these results, along with a downstream application analysis is provided in Appendix C.

4.2.2 Dynamic Fidelity. The physical fidelity of our model was evaluated in terms of trip dynamics. First, we computed the velocity profile distributions based on the trajectories generated by all methods and then derived the corresponding acceleration and jerk via discrete differentiation. Table 2 reports the dynamic distribution shifts relative to those of the real-world trajectories. The results indicate that our proposed method achieves the highest dynamic fidelity compared to the baseline methods. On the Porto dataset, for example, the JSD for all three features of GCDM is only half that of DiffTraj, the current state of the art. While there’s less improvement on the Harbin data, our method still achieves significantly higher fidelity than the next best baseline on acceleration (31.8% better than Diffwave) and jerk (8.7% better than Sashimi).

We also visualized the distributions of the absolute velocity, acceleration, and jerk for trajectories generated by GCDM and the current SOTA (DiffTraj) on the Porto dataset. The histograms are shown in Figure 5, and the corresponding cumulative distribution functions (CDFs) are presented in Figure 6. Note that in Figure 5,

each bin is rescaled horizontally to facilitate comparison across histograms sharing the same value range. For instance, the first three bars in Figure 5a illustrate the proportion of location points with velocities in the range of 0 to 2.333 m/s for the ground truth, GCDM, and DiffTraj. While there is room for further improvement, the proposed method (yellow) aligns better physically with ground-truth data (green) compared with the current SOTA (blue). For example, in both the ground truth dataset and the dataset generated by our method, approximately 30% of the data points exhibit velocities between 0 and 2.333 m/s compared to around 27% for DiffTraj, indicating that GCDM mitigates the error at absolute velocity by around 3% in this bin. The CDFs also corroborate this trend in Figure 6, where the yellow line is closer to the green line than the blue line. Thus, it is reasonable to conclude that the proposed geo-lucid method generates trajectories with higher physical fidelity than existing state-of-the-art approaches.

4.3 RQ2: Sensitivity Analysis

We conducted ablation studies to quantify the contribution of our model’s key components. Results are summarized in Table 3. We began with a baseline approach employing a standard conditional U-Net framework [73] that does not incorporate the road map. Next, we augmented the baseline by adding map information into the model using the geo-lucid latent variable (denoted as *GCDM-v1 (Stage B w. map)*), which yielded a notable performance improvement. We then incorporated spatial hierarchical generation without the map-based correction, as detailed in the third row of Table 3 (namely, *GCDM-v2 (Stage A w/o. map + Stage B w. map)*). The results indicate that omitting the map-based correction in stage A leads to only marginal performance gains (or even performance reduction on the Harbin dataset) compared with the single-stage generation. This is likely due to error accumulation inherent in the cascaded generation process. In the last test, we integrated the map-based projection module into the model to arrive at the complete model (i.e., *GCDM-v3 (Two-stage w. map)*). The additional performance boost validates the efficacy of incorporating map information in both stages of the generation process.

We also tested the effect of grid size on the quality of trajectories generated by our model. We trained GCDM twice on the Porto dataset, once using a grid size of 32×32 in Stage A and once using a grid size of 64×64 in Stage A, and compared the scores on geo-distribution similarity. In each case, the granularity used for evaluation was the same as that used for training. The results are shown in Table 4. Using fewer grids in stage A results in poorer generation quality, though it still outperforms the single-stage version of GCDM-v1 (first row). This observation aligns with the assumption that a more detailed grid in stage A can provide better guidance for the generation in Stage B in GCDM. A similar trend is also evident on the Harbin dataset.

4.4 RQ3: Time Complexity Analysis

To compare the computational cost of the proposed GCDM with existing methods (e.g., DiffTraj), we measured model run/inference time as an indicator of time complexity and examined the trade-off between time complexity and trajectory geo-distribution similarity (density error). In this experiment, we included only DiffTraj as it is

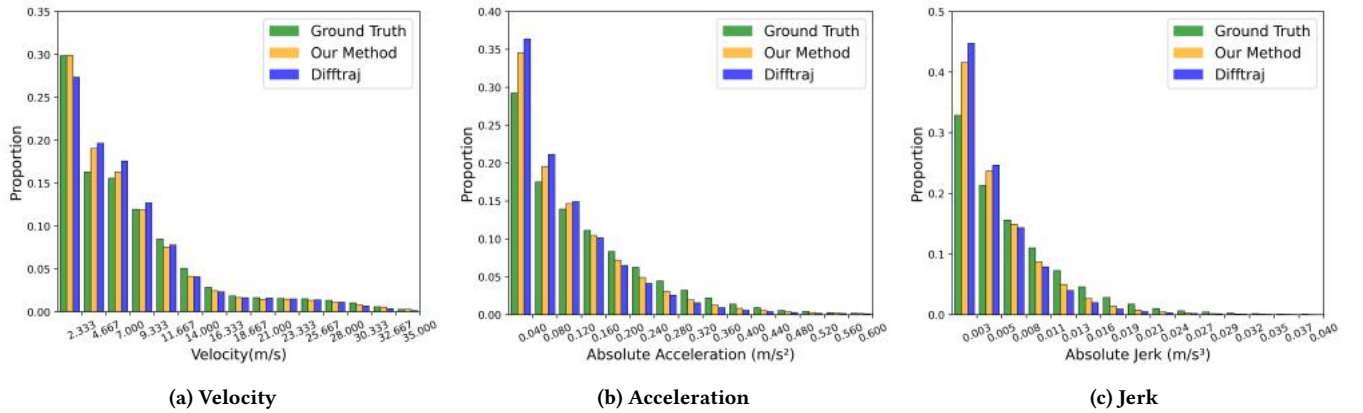
²<https://github.com/yang-mingzhou/GCDM>

Table 1: Comparison of the performance of trajectory generation methods in geo-distribution.

Dataset	Method	Density Error ($\downarrow$)	Trip Error ($\downarrow$)	Pattern Score ($\uparrow$)
Porto	EETG	0.0523	0.1297	0.8495
	Diffwave	0.0275	0.0951	0.7153
	Sashimi	0.0230	0.0810	0.8614
	Difftraj	0.0054	0.0766	0.8566
	Ours	0.0047	0.0736	0.8634
Harbin	EETG	0.0064	0.0217	0.8262
	Diffwave	0.0172	0.0158	0.4700
	Sashimi	0.0163	0.0163	0.4367
	Difftraj	0.0060	0.0130	0.8171
	Ours	0.0046	0.0125	0.8248

Table 2: Comparison of the performance of trajectory generation methods in dynamic fidelity.

Porto					Harbin				
Feature	Method	JSD ($\downarrow$)	Wasserstein ($\downarrow$)	KS-stat ($\downarrow$)	Feature	Method	JSD ($\downarrow$)	Wasserstein ($\downarrow$)	KS-stat ($\downarrow$)
Velocity	EETG	0.0136	2.1303	0.2002	Velocity	EETG	0.0272	3.4354	0.1443
	Diffwave	0.0084	1.1324	0.0929		Diffwave	0.0425	7.4414	0.3105
	Sashimi	0.0152	1.6305	0.1333		Sashimi	0.0666	7.2289	0.3458
	Difftraj	0.0048	0.5761	0.0679		Difftraj	0.0054	1.2581	0.0567
	Ours	0.0022	0.4544	0.0536		Ours	0.0050	1.2886	0.0879
Acceleration	EETG	0.0450	0.0698	0.3825	Acceleration	EETG	0.1002	0.1707	0.3749
	Diffwave	0.0218	0.0369	0.1968		Diffwave	0.0239	0.1662	0.3889
	Sashimi	0.0249	0.0375	0.1985		Sashimi	0.0286	0.1652	0.4219
	Difftraj	0.0121	0.0368	0.1216		Difftraj	0.0257	0.0908	0.1952
	Ours	0.0057	0.0229	0.0846		Ours	0.0163	0.0636	0.1617
Jerk	EETG	0.0589	0.0037	0.4300	Jerk	EETG	0.1389	0.0101	0.4422
	Diffwave	0.0253	0.0022	0.2382		Diffwave	0.0405	0.0094	0.4249
	Sashimi	0.0318	0.0022	0.2353		Sashimi	0.0263	0.0094	0.4592
	Difftraj	0.0196	0.0021	0.0831		Difftraj	0.0372	0.0057	0.2400
	Ours	0.0103	0.0016	0.0609		Ours	0.0240	0.0044	0.1996

**Figure 5: Histograms of the distribution of the absolute velocity/acceleration/jerk of the generated trajectories compared with the ground truth (Porto). Bins are horizontally rescaled to facilitate comparison.**

the current SOTA in geo-distribution similarity. As shown in Table 5, on the Porto dataset, our one-stage method (GCDM-v1) achieves a

7% improvement in density error by incorporating the road map, while incurring a tolerable inference time increase and matching

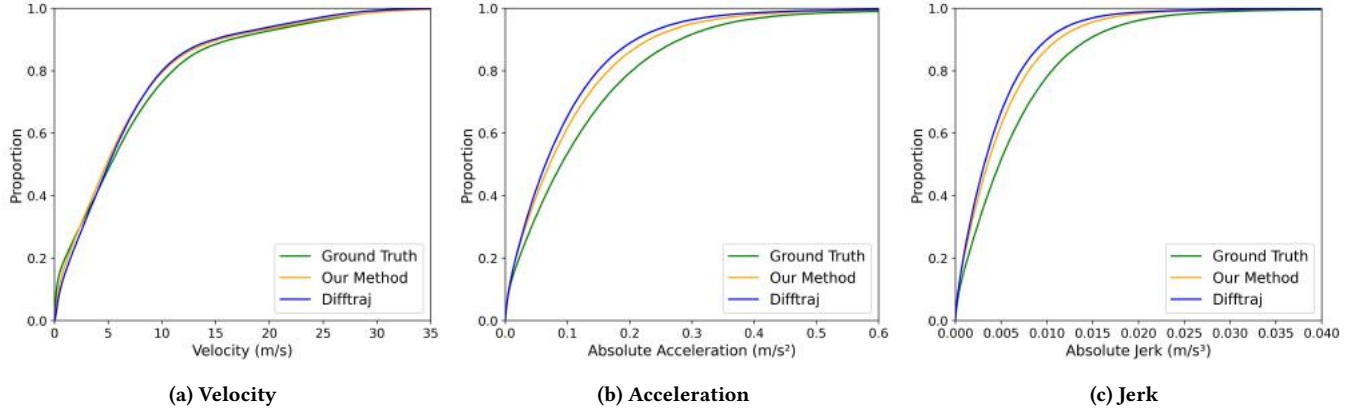


Figure 6: The cumulative distribution functions of the absolute velocity/acceleration/jerk of the generated trajectories compared with the ground truth (Porto).

Table 3: Ablation study of the proposed method.

Dataset	Method	Density Error ($\downarrow$)	Trip Error ($\downarrow$)	Pattern Score ($\uparrow$)
Porto	Baseline	0.0054	0.0766	0.8566
	GCDM-v1 (Stage B w. map)	0.0050	0.0760	0.8597
	GCDM-v2 (Stage A w/o. map + B w. map)	0.0049	0.0757	0.8627
	GCDM-v3 (Two-stage w. map)	0.0047	0.0736	0.8634
Harbin	Baseline	0.0060	0.0130	0.8171
	GCDM-v1 (Stage B w. map)	0.0055	0.0121	0.8199
	GCDM-v2 (Stage A w/o. map + B w. map)	0.0065	0.0128	0.8188
	GCDM-v3 (Two-stage w. map)	0.0046	0.0125	0.8248

Table 4: GCDM’s sensitivity to grid size in stage A.

Dataset	Method	Density Err. ($\downarrow$)	Trip Err. ($\downarrow$)	Pattern Sc. ($\uparrow$)
Porto	GCDM-v1	0.0050	0.0760	0.8597
	GCDM (32)	0.0049	0.0755	0.8616
	GCDM (64)	0.0047	0.0736	0.8634
Harbin	GCDM-v1	0.0055	0.0121	0.8199
	GCDM (32)	0.0052	0.0122	0.8255
	GCDM (64)	0.0046	0.0125	0.8248

DiffTraj’s training time. The other variants (GCDM-v2 and GCDM-v3) further improve geo-distribution similarity by 9–13%, albeit with higher computational cost. Our one-stage method also achieves an 8% improvement in density error compared with SOTA with a similar training/inference time on the Harbin dataset. It can be observed that there is a clear trade-off between time complexity and fidelity of generated trajectories. If downstream tasks prioritize trajectory quality, the full two-stage model (GCDM-v3) significantly outperforms the current state of the art at the cost of longer training and inference times. Conversely, for real-time synthesis, the one-stage version (GCDM-v1) is preferable. The jump in inference time from GCDM-v1 to GCDM-v2 is due to the skip-step inference mechanism (Deep Diffusion Implicit Model) introduced only in Stage B.

5 Broadly Related Work

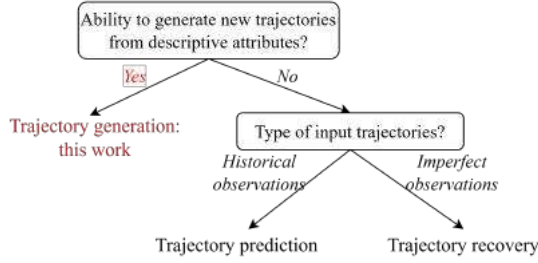
Generative methods for trajectory analysis can be classified into three primary categories of tasks [5, 38], distinguished by their fundamental objectives and interaction with trajectory data, as shown in Figure 7.

Trajectory prediction [16, 32, 35, 41, 65] is typically formulated as a time-series forecasting problem. Its core goal is to use past (historical) observations of a trajectory to estimate its future path. This yields context-aware, short-term forecasts that are grounded in real data. However, the efficacy of prediction models heavily depends on the quality, completeness, and length of the observed trajectory history.

The goal of *trajectory recovery* [42, 47, 63, 74], the second class of methods, is to refine or reconstruct previously recorded but imperfect trajectories, i.e., those that are irregular, sparsely sampled, or contain noisy observations. The objective is to produce a more accurate, complete, or densely sampled representation of the trajectory within its already observed timeline. While invaluable for restoring missing information or correcting inaccuracies, trajectory recovery is inherently limited to improving the fidelity of available input data and cannot infer entirely new paths or segments beyond what can be logically interpolated or corrected from the given observations. Note that ControlTraj [74] generates trajectories given

Table 5: Computational cost versus density error of the proposed method compared with current SOTA.

Dataset	Method	Density Error	Training Time (per epoch)	Inference Time (per sample)
Porto	DiffTraj	0.0054	8.1 min	7.7 ms
	GCDM-v1 (Stage B w. map)	0.0050	8.1 min	8.3 ms
	GCDM-v2 (Stage A w/o. map + B w. map)	0.0049	9.9 + 10.9 min	162.5 ms
	GCDM-v3 (Two-stage w. map)	0.0047	9.9 + 10.9 min	169.8 ms
Harbin	DiffTraj	0.0060	5.3 min	7.3 ms
	GCDM-v1 (Stage B w. map)	0.0055	5.3 min	7.3 ms
	GCDM-v2 (Stage A w/o. map + B w. map)	0.0065	8.7 + 9.6 min	158.0 ms
	GCDM-v3 (Two-stage w. map)	0.0046	8.7 + 9.6 min	167.7 ms

**Figure 7: Decision tree of broadly related work.**

sequences of road segments (i.e., trajectories without timestamps), which means it belongs to the category of trajectory recovery.

In this work, we focus on the problem of *trajectory generation*. This task focuses on designing models that synthesize complex human or object movement patterns by leveraging deep learning to capture the statistical, spatial, and temporal characteristics of real trajectories. Trajectory generation methods have the ability to synthesize new trajectory instances from descriptive attributes (e.g., origin-destination pairs), thus enabling data augmentation in low-coverage areas, albeit risking unrealistic or biased samples.

Trajectory Generation. Related work on trajectory generation falls into two primary categories: rule-based models and data-driven models [25]. In rule-based approaches, trajectories are simulated using predefined parameters (e.g., speed, directional changes, clustering behaviors, etc.) that mirror observed dynamics in the physical world [44]. For example, microscopic traffic simulators like the Simulation of Urban Mobility (SUMO) [27], VISSIM [12], and their variants [33, 68] generate trajectories based on car-following and lane-change models [1]. These approaches leverage comprehensive information from road maps to produce trajectories. However, they often fail to capture the inherent stochasticity of human mobility evident in real-world data. To address this limitation, data-driven deep AI models have been developed for trajectory generation today. Discrete methods simplify the problem by generating trajectories at low spatiotemporal resolution [38], representing them as discretized images [3, 34], grid sequences [13, 30, 37], road segments [7, 23, 60], or points of interest (POIs) [10, 54, 61]. While these representations facilitate computational efficiency, they inherently compromise the fidelity and granularity of the generated trajectories. Several continuous models, such as generative adversarial networks (GANs) [36, 45, 58], variational autoencoders (VAEs) [6, 70], and diffusion models [62, 72, 73], have focused on optimizing the geometric fidelity of synthetic trajectories. However, these

methods ignore the physical laws of trajectory dynamics, limiting their utility in domain-specific scenarios.

Cascaded Generative Model. Cascading pipelines have been investigated in several domains to provide detailed guidance for achieving high-quality results through multi-resolution generation. Examples include text-to-image generation [21, 43], text-to-video generation [59], and audio synthesis [29]. In the area of trajectory generation, prior work has developed *two-stage* generative models that synthesize trajectories at different spatial resolutions [18, 23, 58]. In this work, we explore a novel geo-lucid conditional generative model. Our approach innovates by incorporating rich information from road maps into the generation process at multiple stages, thereby enabling integration across different spatial resolutions.

6 Conclusion and Future Work

The trajectory generation problem aims to synthesize trajectories with high physical fidelity by leveraging historical GPS data and associated descriptive attributes. Current approaches that mostly focus on geometric properties have limited domain applications due to their ignoring of trajectory dynamics. We introduced GCDM, a Geo-lucid Conditional Diffusion Model that integrates the road map attributes with deep generative models. Our experimental evaluations on real-world vehicle trajectory datasets demonstrate that the proposed GCDM method outperforms state-of-the-art techniques in terms of both geo-distribution similarity and physical fidelity.

For future work, we plan to enhance our model by integrating more physics-based knowledge and contextual information (e.g., traffic congestion) to guide trajectory dynamics. We will also incorporate richer road map features—such as connectivity, intersections, and crosswalks—and test the model’s robustness against imperfect road map data (e.g., missing values). Furthermore, we will investigate the model’s generalizability to other data formats, like human mobility data, and assess its utility for a broader range of downstream applications. Finally, we will explore methods to improve privacy preservation in trajectory generation.

Acknowledgments

This material is based upon work supported by the National Science Foundation under Grants No. 1901099, the USDOE Office of Energy Efficiency and Renewable Energy under FOA No. DE-FOA-0002044, and the Minnesota Supercomputing Institute (MSI). We also thank Kim Koffolt and the Spatial Computing Research Group for valuable comments and refinements.

References

- [1] MF Aycin and RF Benekohal. 1999. Comparison of car-following models for simulation. *Transportation research record* 1678, 1 (1999), 116–127.
- [2] Han Bao, Xun Zhou, Yingxue Zhang, Yanhua Li, and Yiqun Xie. 2020. Covid-gan: Estimating human mobility responses to covid-19 pandemic through spatio-temporal conditional generative adversarial networks. In *Proceedings of the 28th international conference on advances in geographic information systems*. 273–282.
- [3] Chu Cao and Mo Li. 2021. Generating mobility trajectories with retained data utility. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2610–2620.
- [4] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman. 2022. Maskgit: Masked generative image transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11315–11325.
- [5] Wei Chen, Yuxuan Liang, Yuanshao Zhu, Yanchuan Chang, Kang Luo, Haomin Wen, Lei Li, Yanwei Yu, Qingsong Wen, Chao Chen, et al. 2024. Deep learning for trajectory data management and mining: A survey and beyond. *arXiv preprint arXiv:2403.14151* (2024).
- [6] Xinyu Chen, Jiajie Xu, Rui Zhou, Wei Chen, Junhua Fang, and Chengfei Liu. 2021. TrajVAE: A Variational AutoEncoder model for trajectory generation. *Neurocomputing* 428 (2021), 332–339.
- [7] Seongjin Choi, Jiwon Kim, and Hwasoo Yeo. 2021. TrajGAIL: Generating urban vehicle trajectories using generative adversarial imitation learning. *Transportation Research Part C: Emerging Technologies* 128 (2021), 103091.
- [8] Junyoung Chung, Kyle Kastner, Laurent Dinh, Kratarth Goel, Aaron C Courville, and Yoshua Bengio. 2015. A recurrent latent variable model for sequential data. *Advances in neural information processing systems* 28 (2015).
- [9] Daniel Delling, Peter Sanders, Dominik Schultes, and Dorothea Wagner. 2009. Engineering route planning algorithms. In *Algorithmics of large and complex networks: design, analysis, and simulation*. Springer, 117–139.
- [10] Bangchao Deng, Xin Jing, Tianyue Yang, Bingqing Qu, Philippe Cudre-Mauroux, and Dingqi Yang. 2024. Revisiting Synthetic Human Trajectories: Imitative Generation and Benchmarks Beyond Datasaurus. *arXiv preprint arXiv:2409.13790* (2024).
- [11] Yuntao Du, Yujia Hu, Zhikun Zhang, Ziquan Fang, Lu Chen, Baihua Zheng, and Yunjun Gao. 2023. Ldptrace: Locally differentially private trajectory synthesis. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1897–1909.
- [12] Martin Fellendorf and Peter Vortisch. 2010. Microscopic traffic flow simulator VISSIM. *Fundamentals of traffic simulation* (2010), 63–93.
- [13] Jie Feng, Zeyu Yang, Fengli Xu, Haisu Yu, Mudan Wang, and Yong Li. 2020. Learning to simulate human mobility. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 3426–3433.
- [14] Karan Goel, Albert Gu, Chris Donahue, and Christopher Ré. 2022. It's raw! audio generation with state-space models. In *International Conference on Machine Learning*. PMLR, 7616–7633.
- [15] Albert Gu, Karan Goel, and Christopher Ré. 2021. Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396* (2021).
- [16] Zhiming Guo, Xing Gao, Jianlan Zhou, Xinyu Cai, and Botian Shi. 2023. Scenedm: Scene-level multi-agent trajectory generation with consistent diffusion models. *arXiv preprint arXiv:2311.15736* (2023).
- [17] Zahid Halim, Aqsa Khan, Muhammad Sulaiman, Sajid Anwar, and Muhammad Nawaz. 2022. On finding optimum commuting path in a road network: a computational approach for smart city traveling. *Transactions on Emerging Telecommunications Technologies* 33, 2 (2022), e3786.
- [18] Lingyun Han. 2023. Enhanced Generation of Human Mobility Trajectory with Multiscale Model. In *International Conference on Neural Information Processing*. Springer, 309–323.
- [19] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).
- [20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [21] Jonathan Ho, Chitwan Saharia, William Chan, David J Fleet, Mohammad Norouzi, and Tim Salimans. 2022. Cascaded diffusion models for high fidelity image generation. *Journal of Machine Learning Research* 23, 47 (2022), 1–33.
- [22] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. 2017. Image-to-Image Translation with Conditional Adversarial Networks. *CVPR* (2017).
- [23] Wenjun Jiang, Wayne Xin Zhao, Jingyuan Wang, and Jiawei Jiang. 2023. Continuous trajectory generation based on two-stage GAN. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4374–4382.
- [24] Xiangjie Kong, Qiao Chen, Mingliang Hou, Azizur Rahim, Kai Ma, and Feng Xia. 2022. RMGen: A tri-layer vehicular trajectory data generation model exploring urban region division and mobility pattern. *IEEE Transactions on Vehicular Technology* 71, 9 (2022), 9225–9238.
- [25] Xiangjie Kong, Qiao Chen, Mingliang Hou, Hui Wang, and Feng Xia. 2023. Mobility trajectory generation: a survey. *Artificial Intelligence Review* 56, Suppl 3 (2023), 3057–3098.
- [26] Zhifeng Kong, Wei Ping, Jiayi Huang, Kexin Zhao, and Bryan Catanzaro. 2021. DiffWave: A Versatile Diffusion Model for Audio Synthesis. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=xFK8Ymz5J>
- [27] Daniel Krajewicz, Jakob Erdmann, Michael Behrisch, and Laura Bieker. 2012. Recent development and applications of SUMO-Simulation of Urban MOBility. *International journal on advances in systems and measurements* 5, 3&4 (2012).
- [28] Solomon Kullback and Richard A Leibler. 1951. On information and sufficiency. *The annals of mathematical statistics* 22, 1 (1951), 79–86.
- [29] Yichong Leng, Zehua Chen, Junliang Guo, Haohe Liu, Jiawei Chen, Xu Tan, Danilo Mandic, Lei He, Xiangyang Li, Tao Qin, et al. 2022. Binauralgrad: A two-stage conditional diffusion probabilistic model for binaural audio synthesis. *Advances in Neural Information Processing Systems* 35 (2022), 23689–23700.
- [30] Siyu Li, Toan Tran, Haowen Lin, John Krumm, Cyrus Shahabi, and Li Xiong. 2024. Geo-Llama: Leveraging LLMs for Human Mobility Trajectory Generation with Spatiotemporal Constraints. *arXiv preprint arXiv:2408.13918* (2024).
- [31] Xiucheng Li, Gao Cong, Aixin Sun, and Yun Cheng. 2019. Learning Travel Time Distributions with Deep Generative Model. In *Proceedings of the 2019 World Wide Web Conference on World Wide Web, WWW 2019, San Francisco, California, May 13-17, 2019*.
- [32] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations (ICLR '18)*.
- [33] Chumeng Liang, Zherui Huang, Yicheng Liu, Zhanyu Liu, Guanjie Zheng, Hanyuan Shi, Kan Wu, Yuhao Du, Fuliang Li, and Zhenhui Jessie Li. 2023. Cblab: Supporting the training of large-scale traffic control policies with scalable traffic simulation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4449–4460.
- [34] Yan Lin, Huaiyu Wan, Jilin Hu, Shengnan Guo, Bin Yang, Youfang Lin, and Christian S Jensen. 2023. Origin-destination travel time oracle for map-based services. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–27.
- [35] Qingze Tony Liu, Danrui Li, Samuel S Sohn, Sejong Yoon, Mubbasir Kapadia, and Vladimir Pavlovic. 2024. TrajDiffuse: A Conditional Diffusion Model for Environment-Aware Trajectory Prediction. In *International Conference on Pattern Recognition*. Springer, 382–397.
- [36] Xi Liu, Hanzhou Chen, and Clio Andris. 2018. trajGANs: Using generative adversarial networks for geo-privacy protection of trajectory data (Vision paper). In *Location privacy and security workshop*. 1–7.
- [37] Qingyue Long, Huandong Wang, Tong Li, Lisi Huang, Kun Wang, Qiong Wu, Guangyu Li, Yanping Liang, Li Yu, and Yong Li. 2023. Practical synthetic human trajectories generation based on variational point processes. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4561–4571.
- [38] Massimiliano Luca, Gianni Barlacchi, Bruno Lepri, and Luca Pappalardo. 2021. A survey on deep learning for human mobility. *ACM Computing Surveys (CSUR)* 55, 1 (2021), 1–44.
- [39] Dennis Luxen and Christian Vetter. 2011. Real-time routing with OpenStreetMap data. In *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (Chicago, Illinois) (GIS '11)*. ACM, New York, NY, USA, 513–516. doi:10.1145/2093973.2094062
- [40] Jacob Menick and Nal Kalchbrenner. 2019. GENERATING HIGH FIDELITY IMAGES WITH SUBSCALE PIXEL NETWORKS AND MULTIDIMENSIONAL UPSCALING. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=HylzTic5Km>
- [41] Amirhossein Nadiri, Jing Li, Ali Faraji, Ghadeer Abuoda, and Manos Papagelis. 2025. TrajLearn: Trajectory Prediction Learning using Deep Generative Models. *ACM Transactions on Spatial Algorithms and Systems* 11, 3 (2025), 1–33.
- [42] Dongmin Park, Junhyeok Kang, Hwanjun Song, Susik Yoon, and Jae-Gil Lee. 2022. Multi-view POI-level cellular trajectory reconstruction for digital contact tracing of infectious diseases. In *2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, 1137–1142.
- [43] Pablo Pernias, Dominic Rampas, Mats Leon Richter, Christopher Pal, and Marc Auberville. 2024. Würstchen: An Efficient Architecture for Large-Scale Text-to-Image Diffusion Models. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=gU58d5QeGv>
- [44] Dieter Pfoser and Yannis Theodoridis. 2003. Generating semantics-based trajectories of moving objects. *Computers, Environment and Urban Systems* 27, 3 (2003), 243–263.
- [45] Jinmeng Rao, Song Gao, Yuhao Kang, and Qunying Huang. 2020. LSTM-TrajGAN: A deep learning approach to trajectory privacy protection. *arXiv preprint arXiv:2006.10521* (2020).
- [46] Ali Razavi, Aaron Van den Oord, and Oriol Vinyals. 2019. Generating diverse high-fidelity images with vq-vae-2. *Advances in neural information processing systems* 32 (2019).
- [47] Huimin Ren, Sijie Ruan, Yanhua Li, Jie Bao, Chuishi Meng, Ruiyuan Li, and Yu Zheng. 2021. Mtrajrec: Map-constrained trajectory recovery via seq2seq multi-task learning. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1410–1419.
- [48] Mats L Richter, Wolf Bytner, Ulf Krumnack, Anna Wiedenroth, Ludwig Schallner, and Justin Shenk. 2021. (Input) size matters for CNN classifiers. In *Artificial Neural*

- Networks and Machine Learning–ICANN 2021: 30th International Conference on Artificial Neural Networks, Bratislava, Slovakia, September 14–17, 2021, Proceedings, Part II* 30. Springer, 133–144.
- [49] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10684–10695.
- [50] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III* 18. Springer, 234–241.
- [51] T-YLPG Ross and GKHP Dollár. 2017. Focal loss for dense object detection. In *proceedings of the IEEE conference on computer vision and pattern recognition*. 2980–2988.
- [52] Shashi Shekhar and Hui Xiong. 2007. *Encyclopedia of GIS*. Springer Science & Business Media.
- [53] Aaron Van Den Oord, Oriol Vinyals, et al. 2017. Neural discrete representation learning. *Advances in neural information processing systems* 30 (2017).
- [54] Huandong Wang, Changzheng Gao, Yuchen Wu, Depeng Jin, Lina Yao, and Yong Li. 2023. PateGail: a privacy-preserving mobility trajectory generator with imitation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 14539–14547.
- [55] Jingyuan Wang, Jiawei Jiang, Wenjun Jiang, Chao Li, and Wayne Xin Zhao. 2021. Libcity: An open library for traffic prediction. In *Proceedings of the 29th international conference on advances in geographic information systems*. 145–148.
- [56] Jinzhong Wang, Xiangjie Kong, Feng Xia, and Lijun Sun. 2019. Urban human mobility: Data-driven modeling and prediction. *ACM SIGKDD explorations newsletter* 21, 1 (2019), 1–19.
- [57] Sheng Wang, Zhifeng Bao, J Shane Culpepper, and Gao Cong. 2021. A survey on trajectory data management, analytics, and learning. *ACM Computing Surveys (CSUR)* 54, 2 (2021), 1–36.
- [58] Xingrui Wang, Xinyu Liu, Ziteng Lu, and Hanfang Yang. 2021. Large scale GPS trajectory generation using map based on two stage GAN. *Journal of Data Science* 19, 1 (2021), 126–141.
- [59] Yaohui Wang, Xinyuan Chen, Xin Ma, Shangchen Zhou, Ziqi Huang, Yi Wang, Ceyuan Yang, Yinan He, Jiahuo Yu, Peiqing Yang, et al. 2024. Lavie: High-quality video generation with cascaded latent diffusion models. *International Journal of Computer Vision* (2024), 1–20.
- [60] Yong Wang, Guoliang Li, Kaiyu Li, and Haitao Yuan. 2022. A deep generative model for trajectory modeling and utilization. *Proceedings of the VLDB Endowment* 16, 4 (2022), 973–985.
- [61] Yu Wang, Tongya Zheng, Shunyu Liu, Zunlei Feng, Kaixuan Chen, Yunzhi Hao, and Mingli Song. 2024. Spatiotemporal-Augmented Graph Neural Networks for Human Mobility Simulation. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [62] Tonglong Wei, Youfang Lin, Shengnan Guo, Yan Lin, Yiheng Huang, Chenyang Xiang, Yuqing Bai, and Huaiyu Wan. 2024. Diff-rntraj: A structure-aware diffusion model for road network-constrained trajectory generation. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [63] Tonglong Wei, Youfang Lin, Yan Lin, Shengnan Guo, Lan Zhang, and Huaiyu Wan. 2024. Micro-Macro Spatial-Temporal Graph-Based Encoder-Decoder for Map-Constrained Trajectory Recovery. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [64] OpenStreetMap Wiki. 2024. Main Page — OpenStreetMap Wiki.. https://wiki.openstreetmap.org/w/index.php?title=Main_Page&oldid=2752444 [Online; accessed 8-February-2025].
- [65] Yuan Xu, Jiajie Xu, Jing Zhao, Kai Zheng, An Liu, Lei Zhao, and Xiaofang Zhou. 2022. Metapt: An adaptive meta-optimized model for personalized spatial trajectory prediction. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2151–2159.
- [66] Yiyuan Yang, Ming Jin, Haomin Wen, Chaoli Zhang, Yuxuan Liang, Lintao Ma, Yi Wang, Chenghao Liu, Bin Yang, Zenglin Xu, et al. 2024. A survey on diffusion models for time series and spatio-temporal data. *arXiv preprint arXiv:2404.18886* (2024).
- [67] Taeyoung Yun, Sujin Yun, Jaewoo Lee, and Jinkyoo Park. 2024. Guided trajectory generation with diffusion models for offline model-based optimization. *arXiv preprint arXiv:2407.01624* (2024).
- [68] Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. 2019. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The world wide web conference*. 3620–3624.
- [69] Junbo Zhang, Yu Zheng, and Dekang Qi. 2017. Deep spatio-temporal residual networks for citywide crowd flows prediction. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 31.
- [70] Liming Zhang, Liang Zhao, and Dieter Pfoser. 2022. Factorized deep generative models for end-to-end trajectory generation with spatiotemporal validity constraints. In *Proceedings of the 30th International Conference on Advances in Geographic Information Systems*. 1–12.
- [71] Zijian Zhang, Zhou Zhao, Jun Yu, and Qi Tian. 2023. Shiftddpm: Exploring conditional diffusion models by shifting diffusion trajectories. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 3552–3560.
- [72] Yuanshao Zhu, Yongchao Ye, Ying Wu, Xiangyu Zhao, and James Yu. 2023. Synmob: Creating high-fidelity synthetic gps trajectory dataset for urban mobility analysis. *Advances in Neural Information Processing Systems* 36 (2023), 22961–22977.
- [73] Yuanshao Zhu, Yongchao Ye, Shiyao Zhang, Xiangyu Zhao, and James Yu. 2023. Difftraj: Generating gps trajectory with diffusion probabilistic model. *Advances in Neural Information Processing Systems* 36 (2023), 65168–65188.
- [74] Yuanshao Zhu, James Jianqiao Yu, Xiangyu Zhao, Qidong Liu, Yongchao Ye, Wei Chen, Zijian Zhang, Xuetao Wei, and Yuxuan Liang. 2024. Controltraj: Controllable trajectory generation with topology-constrained diffusion model. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4676–4687.

A A Detailed Look at GCDM Framework

A.1 Pretraining VAE

Neural Architecture. We pre-trained a Vector Quantised-Variational AutoEncoder (VQ-VAE) [53] to encode the trajectory image in Stage A. The VQ-VAE comprises an encoder, a quantization bottleneck, and a decoder. The encoder begins with a 3×3 convolution (stride 1, padding 1) to produce 32 feature channels, followed by two DownBlocks that successively increase the channel dimensions (from 32 to 64, then from 64 to 128) while performing spatial downsampling with 4×4 convolutions (stride 2, padding 1). A mid-level block then refines the latent representation via dual residual pathways and a multi-head attention module (16 heads) before reducing the feature channels to 3 through a 3×3 convolution and a 1×1 convolution, thereby preparing the output for quantization using a 2048-entry codebook. The decoder mirrors the encoder by first projecting the quantized 3-channel input to 128 channels with a 3×3 convolution, then processing it through an analogous mid-level block and two UpBlocks that sequentially reduce the number of channels from 128 to 64 and then from 64 to 32 using transposed convolutions (4×4 , stride 2, padding 1), reconstructing the image with a final GroupNorm and 3×3 convolution to produce output.

Training Details. The VQ-VAE model was trained using the AdamW optimizer with a learning rate of $1e-5$ and a batch size of 512 for 30 epochs. Each input binary image had the shape $1 \times 64 \times 64$. The downsampling sizes were $(32 \rightarrow 64 \rightarrow 128)$ and the mid-level channel sizes $(128 \rightarrow 128)$, with one layer used in each of the down, mid, and upsampling stages. The training regimen included a combination of reconstruction loss, codebook loss, and commitment loss [53]. The reconstruction loss was defined as in Equation (13) with $\alpha = 0.05$ and $\gamma = 7$. The scale weight for codebook loss is one, and that for commitment loss is 0.2. We also included a discriminator loss from a PatchGAN [22] after 1000 steps, weighted by a factor of 0.5.

A.2 Stage A

Neural Architecture. We adopted a conditional U-Net architecture from Diffraj [73] for latent variable denoising in Stage A. The trajectory properties were encoded through a WideAndDeep embedding framework, where numerical features (total distance, total travel time, trajectory length, average distance, average velocity) were processed via a wide fully-connected layer to a 512-dimensional space. Discrete categorical features (departure time slots, start locations, end locations) are embedded into 256-dimensional vectors before being concatenated and processed through two fully-connected layers ($768 \rightarrow 512 \rightarrow 512$) to capture complex feature interactions. These embeddings are incorporated into the diffusion process by combining them with diffusion step embeddings. Specifically, we employ Sinusoidal embedding to represent each diffusion step t as a 128-dimensional vector and then apply two shared-parameter fully connected layers. The conditioning signals are then injected into each Down and Up block of a U-Net [50] backbone through dedicated embedding layers (SiLU activation followed by a linear projection to match the respective block’s channel dimension) and added to the feature maps after the primary convolution operations. The U-Net backbone processes the 3-channel grid

trajectory input through an initial DoubleConv block ($3 \rightarrow 64$ channels) with 3×3 Conv $\rightarrow$ GroupNorm $\rightarrow$ GELU $\rightarrow$ Conv $\rightarrow$ GroupNorm [19]. The encoder pathway comprises three Down blocks with max-pooling operations, progressively expanding feature dimensions ($64 \rightarrow 128 \rightarrow 256 \rightarrow 256$) through DoubleConv operations, while interleaved self-attention modules at resolution 16 capture long-range dependencies. Two bottleneck DoubleConv blocks (256 channels) refine the latent representation before the decoder pathway employs three Up blocks with bilinear upsampling, gradually reducing feature dimensions ($512 \rightarrow 128 \rightarrow 64 \rightarrow 64$) through DoubleConv operations, with skip connections from corresponding encoder levels, ultimately culminating in a 1×1 convolution that produces a 3-channel output.

Training Details. We trained the latent diffusion model with a batch size of 1024 for 200 epochs. The model was trained using the AdamW optimizer with a learning rate of $1e-5$. The diffusion process operated over 1000 timesteps, employing a beta schedule that increased linearly from 0.0015 to 0.0195. Regularization was achieved through a dropout rate of 0.1, while an exponential moving average (EMA) with a rate of 0.9999 was applied.

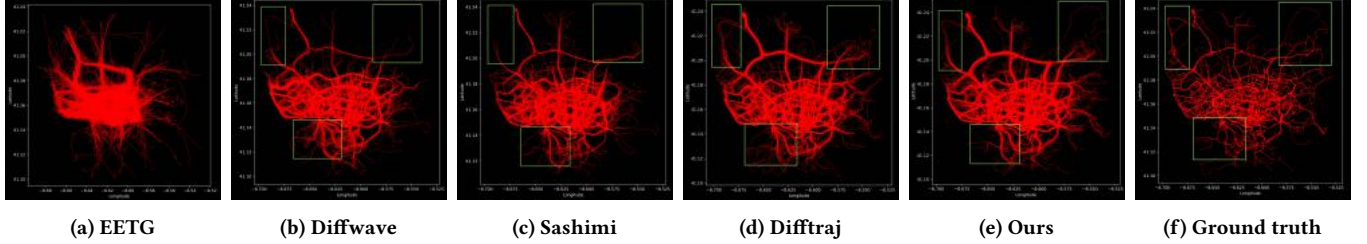
A.3 Stage B

Neural Architecture. In stage B, we adopted the same denoising model for residual diffusion as in stage A. We incorporated the latent gridded trajectory from Stage A through the GridEncoder module, as described in Section 3.2.2, which processes the 2D latent trajectory through two strided convolutional layers ($3 \rightarrow 64 \rightarrow 64$ channels) with ReLU activations and an adaptive average pooling layer, producing a fixed 8×8 spatial representation. The resulting features are reshaped and permuted, with learned 2D positional embeddings added to retain spatial information. The main backbone processes features in 1D space, initiated by a Conv1d layer that maps 2-dimensional coordinates to 128 channels. The encoder pathway consists of four down-sampling blocks with ResNet architectures ($128 \rightarrow 256$ channels), where each block contains dual ResNet modules with GroupNorm, temporal embedding injection (512-dimensional), and dropout ($p=0.1$). The spatial cross-attention modules introduced in Section 3.2.2 are inserted after each down-sampling block (one module processing 128-channel features in the first block, followed by three modules handling 256-channel features), with symmetric placement in the decoder path. These modules project queries to a 64-dimensional space before cross-attending to the grid-encoded tokens. The middle block features a ResNet block, a self-attention module, another ResNet block, and a spatial cross-attention module, all operating on 256-channel features. The decoder pathway employs four up-sampling blocks with ResNet modules, progressively reducing feature dimensions ($384 \rightarrow 128$ in the first block, $512 \rightarrow 256$ in subsequent blocks) through skip connections before a final Conv1d layer produces the 2-dimensional coordinate output. The model employs the same WideAndDeep attribute embedding framework as Stage A to maintain consistency in trajectory property conditioning.

Training Details. We trained the model with a batch size of 1024 over 200 epochs using the AdamW optimizer with a learning rate of $2e-4$. We decomposed the trajectory using $\lambda = 0.1$ per

Table 6: Statistics of the Trajectory Datasets.

Dataset	Lon/lat Bounding box	Area (km)	Trajectory Number	Average Time (min)	Average Distance (km)
Porto	[-8.696214, -8.526609, 41.0994, 41.240736]	14.20×15.72	1,411,784	12.32	5.33
Harbin	[126.50613, 126.77186, 45.65792, 45.830902]	20.62×19.23	1,250,071	25.39	9.13

**Figure 8: Geographic visualization of generated trajectories.**

Equation (15) to facilitate residual diffusion. The diffusion process was parameterized with 500 timesteps and a linear beta schedule ranging from 0.0001 to 0.05. We employed a dropout rate of 0.1 across all layers and an EMA of model parameters with a decay rate of 0.9999. We adopted a stochastic noise augmentation strategy with 100-step noising to the latent gridded trajectories [21], enabling the model to handle imperfect embeddings during inference.

B Experiment and Setup

Dataset. We evaluate the performance of the proposed method on two distinct taxi trajectory datasets, one from Porto, Portugal, and the other from Harbin, China. The Porto dataset comprises trajectories collected from 442 taxis between July 1, 2013, and June 30, 2014, while the Harbin dataset contains data recorded from 1,3000 taxis between January 3 and January 7, 2015. Both datasets were preprocessed by applying noise filtering [57] and then removing trajectories that were either too short (i.e., less than 5 minutes) or excessively long (i.e., greater than 80 km). The statistical properties of the cleaned datasets are summarized in Table 6. Moreover, since the U-Net architecture requires fixed-size inputs, we uniformly sampled each trajectory to a fixed length during the generation process and then reconstructed the original resolution using linear interpolation during evaluation.

Evaluation Metrics. To assess the performance of the trajectory generation models, we generated 3,000 synthetic trajectories using each model and averaged the evaluation results over 10 iterations to ensure robust performance estimates. To quantify the geo-distribution similarity between generated trajectories and ground truth data, we discretized each urban area into a 16×16 grid and computed the frequency distribution of trajectory points within each cell. We employed three complementary metrics to capture different aspects of the geo-distribution similarity: **(1) Density Error**, which quantifies the global similarity between the geographic distributions of generated and original trajectories. It is defined as the Jensen-Shannon divergence (JSD) between the gridded distribution matrices. **(2) Trip Error**, which evaluates trajectory accuracy at the individual journey level by analyzing origin-destination patterns. It computes the probability distributions of start and end points

for both the original and generated trajectories and measures their similarity using the Jensen-Shannon divergence, thereby capturing how well the generated trajectories preserve the spatial characteristics of complete journeys. **(3) Pattern Score**, which assesses semantic-level similarity by identifying the top- n most frequently visited grid cells in the trajectories. Denoting the original and generated pattern sets as P and P_{gen} respectively, the pattern score is computed as the harmonic mean of precision and recall [73].

$$\text{Pattern score} = \frac{2 \times \text{Precision}(P, P_{\text{gen}}) \times \text{Recall}(P, P_{\text{gen}})}{\text{Precision}(P, P_{\text{gen}}) + \text{Recall}(P, P_{\text{gen}})} \quad (24)$$

C Extra Experiments

Visualization Analysis of Geo-Distribution. Figure 8 illustrates the geographical distribution of trajectories generated on the Porto dataset. Although all diffusion-based models capture a broad geographic overview, GCDM yields finer details. In particular, trajectories generated by GCDM exhibit closer alignment with the underlying road network, especially in complex or sparsely sampled regions (highlighted by green boxes). For instance, GCDM uniquely generates a long trajectory in the upper right region and produces trajectories in the lower green box that are less noisy and more consistent with road geometries. These qualitative observations further validate the quantitative improvements, confirming that GCDM offers a robust solution for high-fidelity trajectory generation.

Downstream application. We conducted a downstream analysis of the generated dataset on grid-based traffic in/out flow prediction to assess the data utility of the synthetic trajectories, a key application of trajectory data analysis. We trained an STResNet [69] (via LibCity [55]) on three datasets: (i) the original real-world Porto trajectories; (ii) trajectories generated by our proposed GCDM; and (iii) trajectories generated by the state-of-the-art DiffTraj. To quantify performance degradation when training on synthetic data, we computed the relative error difference in both MSE and RMSE against the model trained on ground-truth data. Models trained on GCDM-generated trajectories exhibit only a 7.3% increase in MSE and a 3.7% increase in RMSE, compared to 15.3% and 7.9% increases for DiffTraj, thereby validating the downstream utility of GCDM.